Zooming in on Decompiler Benchmarking

A Framework for Statement-Level Instrumentation and Diagnostic Feedback

by

John Chen, 851585377

Course code:	IB9902	
Thesis committee:	Dr. Nico Naus,	Open Universiteit
	Dr. Tim Steenvoorden,	Open Universiteit

Open Universiteit
de beste! www.ou.nl

Zooming in on Decompiler Benchmarking

A Framework for Statement-Level Instrumentation and Diagnostic Feedback

A thesis submitted in partial fulfillment of the requirements for a bachelor's degree in Computer Science

John Chen, 851585377

Open Universiteit, The Netherlands

***Abstract*—Decompilation in reverse engineering strives to recover information lost during compilation and reconstruct representative source code. To measure how decompilers perform, developers rely on proprietary benchmarks, hindering objective comparison between decompilers. While frameworks like Deb'm and Kline's framework introduced independent benchmarks to address this problem, these approaches are limited to specific test sets or decompilers. Generalization of measured performance on artificial test sets to realistic circumstances remains an open issue. Real-world benchmarks like DecompileBench provide holistic summaries that miss subtle details in performance required for more actionable feedback. We address these limitations through a proof-of-concept framework, inspired by Deb'm, that enables statement-level ground truth on real-world and arbitrary test sets. This allows developers to more closely inspect and evaluate the performance of their decompilers. Finally, we showcase such inspection by benchmarking Ghidra on source code reconstruction and use concrete metrics to analyze, compare, and interpret its performance across testing environments.**

***Index Terms*—Software Engineering, Decompilation, Code-markers, Instrumentation, Benchmarking.**

I. Introduction

Software facilitates many of our services and as a result software incidents can prove costly with lasting effects on society or personal lives. Resolving some of these incidents requires the inspection of legacy or foreign binaries, where the source code is not always available. In order to mitigate the costs of such events, security specialists dedicate time to understand and recover readable forms of lost or unknown code through reverse engineering. This process requires expertise and is aided by decompilers [1] to obtain more comprehensible higher level representations of binaries. It is important that reverse engineers can trust the results of their decompilers. To ascertain the quality of a decompiler and its algorithms, different benchmarks and metrics exist. Transparent and informative sets of metrics are important [2] in order to better inform developers.

Benchmarks are used to screen for decompiler features that require attention or to give diagnostic feedback. Measurements can be composed of multitudes of features, but not every benchmark chooses to report on the subscores of every feature. Historically, benchmarks are used to compare features across decompilers. For developers, benchmarks have increasingly taken a role as a guide towards improvements in their own decompiler algorithms. However, the requirements of diagnostically inclined benchmarks are more stringent, which are accompanied by various forms of implementation couplings. Such coupling has so far accompanied numerous different benchmarks, each tailored to measuring an aspect of decompilation. The drawback of numerous benchmarks and their dependencies is that they complicate selection, comparison and metric comprehension across benchmarks.

Holistic benchmarks [3, 4, 5] and metrics [6, 7] can signal areas of attention, but they are limited in conveying nuances between decompilers. For similar scores, holistic overviews obfuscate subtle differences in performance in multitudes of aspects. For example, R2I [6] condenses the evaluation of 31 features into a single comparative score.

Furthermore, metrics such as "readability" are open to interpretation, causing confusion when measured through different, complicated methodologies in various benchmarks. In addition, representativeness and reproducibility remain issues typical of subjective criteria [2].

Such metrics can seem somewhat arbitrary, and their relation to decompilers in reverse engineering development is unclear.

Alternatively, existing granular benchmarks [8, 9, 10] avoid some of this ambiguity, but are restricted to specific decompilers, architectures, IR representations, or artificial test sets, which hinders benchmarking. Table I shows how these factors compound: benchmarks overlap in readability metrics and in function- and source-level constructs, while differing in their restrictions to artificial or custom code.

Current solutions are therefore restricted to artificial code, lack transparent granular metrics, or require complicated evaluation procedures.

We propose a two-pronged solution to enable both granular transparent metrics and more inclusive test sets, using only the source code context.

First, we detach the code generation from the benchmarking methodology, which allows us to use both real-world and synthetic code.

Second, we develop a method that addresses the limitations of holistic feedback by increasing the granularity of benchmarks on decompilers for more transparent feedback.

TABLE I: Overview of decompiler benchmarks.

Benchmark	Scope	Dataset Granularity	Key Metrics	Implementation
CodeFuse-DeBench [5] (2026)	To measure readability, recompilability, and functional equivalence.	Function-level: custom built dataset.	Readability, recompilability, and functionality.	Large Language Model-dependent evaluation. Binary analysis using "Frida hook" on successful recompilation.
DecompileBench [4] (2025)	To measure key metrics over more representative "real-world" code and to inform security experts on decompiler incorporation.	Function-level: fuzzer activated functions extracted through OSS-Fuzz.	Readability, helpfulness, and readability + helpfulness.	Fuzzer dependent function selection + evaluation.
Decompile-Bench [3] (2025)	To construct a comprehensive data set of real-world code and explore its effects on Large Language Models with existing metrics	Function-level: functions extracted from GitHub repositories.	Re-Executability, R2I, and edit similarity.	Function-level binary-source pairing and evaluation through custom CTF framework.
D-Helix [10] (2024)	Measures areas of decompilation errors and gives support towards decompilation correctness.	Binaries and object files from GitHub	Function prototype recovery, literal value recovery, type recovery, structure recovery, etc.	Evaluation based on SymDiff with compiled and decompiler IR, and dynamic modifications with a custom TUNER program.
Deb'm [9] (2024)	Framework for evaluating language constructs using statement-level ground truth	Statement and IR-level	Source-level program constructs	Granular benchmark on custom generated code with LLVM-IR dependency.
Kline's benchmark [8] (2022)	Framework for quantifying and assessing the accuracy of decompilers regarding high-level program constructs.	DWARF, binary-source mapping on instruction-level	Source-level program constructs	Evaluates using custom framework for comparison to ground truth. Ghidra only, with potential for decompiler expansions.

Our contributions are as follows:

- **A novel statement-level instrumentation mechanism designed to be code agnostic.** The mechanism creates ground truths on individual statements from real-world and synthetic code for benchmarking.
- **A granular prototype benchmarking tool.** We use the instrumentation mechanism to enable a diagnostic form of benchmarking that can provide concrete feedback on decompilation results.

The prototype implementation is available on GitHub[1] . We start this thesis with a general overview of the prototype by describing the framework in Section II, followed by the discussion of the proof-of-concept methodology in Section III. We then demonstrate the implications of concrete granular benchmarks by describing benchmark examples in Section IV, how we validate the instrumentation in Section V, followed by limitations in Section VI.

We review our work against previous literature in Section VII, summarize our contributions in Section VIII, and outline future research directions in Section VIII-A.

[1]https://github.com/JCAtGit/Castle

II. Benchmark Framework

Van den Bos et al. and Theodoridis et al. [9, 11] show that embedded ground truths within source code can persist through compilation and decompilation under specific optimization conditions. The C language contains constructs that allow it to instruct compiler behavior regarding optimization within the language. By constraining the instrumentation to solely the source code context, the ground truth can be decoupled from compiler and binary conditions. This has advantages over the ground truth information from sources such as DWARF [12], where debug information inserted by compilers can sometimes contain bugs [13]. Source embedded ground truths can still be available after decompilation and are relatively simple to extract. In comparison DWARF is consumed during decompilation and does not map between the decompiled source and the original source code. In addition, diverse build processes within repositories complicate the retrieval of DWARF information. These concepts form the basis of our framework.

A. Instrumentation Techniques

The C language allows the following constructs to communicate optimization instructions to the compiler from within the source:

Markers Special identifiable statements [9, 11] that keep their relative position to other statements and persist through the (de)compilation pipeline (Listing 1 and 2).

Attributes C language attributes [14, 15, 16] that influence compiler behavior for constructs that allow enhancements through attributes.

Preprocessor directives Text in code that gets processed by preprocessors before compilation [17]. In particular directives that instruct compiler optimization behavior.

The markers are inserted next to existing statements, attributes are appended onto existing functions and headers containing preprocessor directives are added to source files. We refer to this instrumentation mechanism, of marker and attribute "graft" enhancements, as Mage.

The framework consists of the following components:

Mage The instrumentation artifact.

MageEval Extractor of ground truth mappings.

MageEvalAnalyzer Summarizer of extracted results.

MageEvalComparer Comparison tool for summaries of different test sets.

Architecture and Scaffolders A set of supporting binaries that integrate the tool set and automate the benchmarking process.

We first proceed by describing how Mage instruments code in Section II-B, followed by an explanation of how results are obtained after (de)compilation in Section II-C. We then describe how test set results are setup for comparison in Section II-D and how the processes can integrate into a pipeline in Section II-E.

B. The Instrumenter, Mage

Mage's mechanisms allow it to map between the precompilation and post-decompilation source code (Listing 1 and 2). Each statement is "marked" by the preceding "mage_alpha" statement which carries over onto the decompiled source. Functions and files are modified to instruct the compiler to perform the desired optimization passes during compilation. Mage's instrumentations are designed to direct compiler behavior separately for markers.

The marker definitions are defined with "memory clobbers" [18, 19], statements that prevent other statements that read or write to memory from reordering across them, inside the function bodies to enforce statement boundaries. This informs the compiler to not rearrange the positions of such statements relative to each other. Furthermore, the marker definitions are enhanced with attributes that instruct the compiler not to remove, modify and inline markers (Listing 3).

```
int __attribute__((optimize("O0"), optnone))
main (int argc, char* argv[]) {
  mage_alpha0(0);
  int i;
  mage_alpha1(1);
  int print_hash_value = 0;
  mage_alpha2(2);
  if (argc == 2 && strcmp(argv[1], "1") == 0)
    print_hash_value = 1;
  mage_alpha3(3);
  platform_main_begin();
  mage_alpha4(4);
  crc32_gentab();
  mage_alpha5(5);
  func_1();
  mage_alpha6(6);
  for (i = 0; i < 6; i++) {
    transparent_crc(
      g_2[i].f0, "g_2[i].f0",
      print_hash_value
    );
    mage_alpha7(7);
    if (print_hash_value)
      printf("index = [%d] \n", i);
  }
  platform_main_end(
    crc32_context ^ 0xFFFFFFFFUL,
    print_hash_value
  );
  return 0;
}
```

Listing 1: Mage instrumented source code with highlighted mage_alpha markers and attribute instrumentations.

```
undefined8 main(int param_1,long param_2) {
  int iVar1;
  int local_20;
  uint local_1c;
  mage_alpha0(0);
  mage_alpha1(1);
  local_20 = 0;
  mage_alpha2(2);
  if (param_1 == 2) {
    iVar1 = strcmp(*(char **)(param_2 + 8),"1");
    if (iVar1 == 0) {local_20 = 1;}
  }
  mage_alpha3(3);
  platform_main_begin();
  mage_alpha4(4);
  crc32_gentab();
  mage_alpha5(5);
  func_1();
  mage_alpha6(6);
  for (
      local_1c = 0; (int)local_1c < 6;
      local_1c = local_1c + 1) {
    transparent_crc(
      (long)*(char *)(&g_2 + (int)local_1c),
      "g_2[i].f0",local_20
    );
    mage_alpha7(7);
    if (local_20 != 0) {
      printf("index = [%d] \n",(ulong)local_1c);
    }
  }
  platform_main_end(
    crc32_context ^ 0xffffffff,
    local_20
  );
  return 0;
}
```

Listing 2: Instrumented and decompiled source code with highlighted mage_alpha markers.

```
__attribute__((
noinline, used, noipa, optnone, optimize("-O0")
))
```

Listing 3: Marker definition attributes for persistence.

These enhancements allow markers to "stick" to their relative positions regarding their marked statements (markee), which we use as our ground truth.

Benchmarking commonly involves measuring across various optimization levels. Codebases can have custom build processes that can make setting optimization flags complicated, we sidestep this issue by configuring this option through the source code itself. For each instrumented source file we add a header inclusion to that file that controls optimizations locally (Listing 4).

```
#pragma once
#if defined(__clang__)
#pragma clang optimize off
#elif defined(__GNUC__)
#pragma GCC optimize ("O0")
#elif defined(_MSC_VER)
#pragma optimize("", off)
#endif
```

Listing 4: Header macros that control optimization.

In addition we can control optimizations for functions separately by modifying attributes in a comparable manner to allow the survival of markers within higher optimization settings.

Each marker is defined separately with an index embedded within the function declaration and argument. Decompilers, e.g. Ghidra, do not always resolve arguments correctly, resulting in placeholder values. We mitigate this by embedding the index within the name declaration.

Before markers are inserted, we check the surrounding sibling statements for the absence of macros and the presence of a parent block (`CompoundStmt`), we then proceed to insertion.

When the instrumented source code, which we will refer to as "oracle" (Listing 1), is subsequently compiled and decompiled, we obtain instrumented decompiled code (Listing 2), referred to as "decom".

C. Extractor of ground truth mappings, MageEval

After the codebase is instrumented, compiled and decompiled, we traverse the abstract syntax tree (AST) of both the oracle and decom context with the Clang recursive visitor. We extract "marker findings" from the AST—information that characterizes the markee. The information we extract is available through the native Clang API present within the Clang tool set, with minimal custom methods.

This procedure functions as follows:

1) We have the Clang visitor only visit the `CallExpr` class, this class represents functions calls like the marker statement.

```
"2": {
    "decom": [
      {
        "ASTNode": "IfStmt",
        "Source": "... /3.c (489, 3) ",
        "properties": {
          "cond_expr_types": [
            {
              "is_const": false,
              "is_pointer": false,
              "type": "_Bool"
            }
          ],
          "cond_stmt_class": "BinaryOperator",
          "has_cond_expr": true,
          "has_cond_var": false,
          "has_else": false,
          "has_markee": true
        }
      }
    ],
    "oracle": [
      {
        "ASTNode": "IfStmt",
        "Source": "... /3.c (62, 5) ",
        "properties": {
          "cond_expr_types": [
            {
              "is_const": false,
              "is_pointer": false,
              "type": "_Bool"
            }
          ],
          "cond_stmt_class": "BinaryOperator",
          "has_cond_expr": true,
          "has_cond_var": false,
          "has_else": false,
          "has_markee": true
        }
      }
    ]
  }
```

Listing 5: Two findings for marker 2 from Listings 1 and 2.

2) Upon the visit of a `CallExpr`, we verify if the `CallExpr` is a marker through its name property that is immediately derivable through the AST visitor.
3) If the `CallExpr` is found to be a marker, we try to retrieve the markee.
4) If the markee is not another marker, which can happen if markee statements get lost, we proceed to gather characteristics and store it under that marker index. Otherwise we note the results as a recovery loss.

Each marker finding is recorded to a JSON object and stored under an array corresponding to a marker index entry and its source context (Listing 5). For example, when we find an `IfStmt` following `mage_alpha2(2)` in Listing 1, we add the markee `IfStmt` with its details to a record under index 2.

Because the marker findings are stored under a collective index, we have achieved the mapping of the marker findings across both contexts. After each file is processed for marker findings, we proceed to produce "evaluation entries".

This procedure is as follows:

1) For each marker index and finding under decom, a

```
"2": [
    {
      "markee_construct": "IfStmt",
      "parity_cond_expr_types": true,
      "parity_cond_stmt_class": true,
      "parity_cond_var_stmt_class": true,
      "parity_cond_var_types": true,
      "parity_has_cond_expr": true,
      "parity_has_cond_var": true,
      "parity_has_else": true,
      "recovered_construct": true
    }
]
```

Listing 6: An evaluation entry of marker 2.

comparison is started to the related oracle finding, each such finding pair (FP) comparison we refer to as "FP comparison".

2) Within the FP comparisons, the recordings proceed in two ways. If the construct matches, the `recovered_construct` property is set to `true` in an evaluation entry, and its attributes are evaluated (Listing 6). Else if the construct does not match, the `recovered_construct` property and all other attributes are set to `false`.
   Each record contains the property results of a single FP comparison (Listing 6), which are then collected within a file.

D. Summarizer for the extracted results, MageEvalAnalyzer, and the summary aggregator of different codebases, MageEvalComparer

For each of the evaluation entries the construct and its attributes are counted and recorded in an overall local summary for a codebase by MageEvalAnalyzer.

After each entry has been processed, the recovery proportion `P` is then calculated for each construct:

$$\texttt{P = Recovered / FP Comparison} \quad (1)$$

The results are stored at a directory related to the codebase. Finally, for each of the codebase summaries belonging to a test set, an aggregated summary is produced by MageEvalComparer.

MageEvalComparer iterates over all the included codebases and sums the results to a single summary representing that test set.

E. The Pipeline Architecture

The framework uses "scaffolder" executables, helper programs that prepare the activities in each phase, in its architecture (Figure 1) to facilitate the processes associated with different phases; each scaffolder loads configuration files that tell it what tasks are scheduled and how they should be invoked. A task can consist of invoking a code generator, cloning repositories, compilation, decompilation or benchmarking.

CodebaseScaffolder The "Codebase" is a collection of repositories or source code material from different origins. There is no requirement with Mage that excludes any repository other than the usage of C-language. In this phase, the scaffolder generates code, e.g. CSmith or clones repositories for use further down the pipeline.

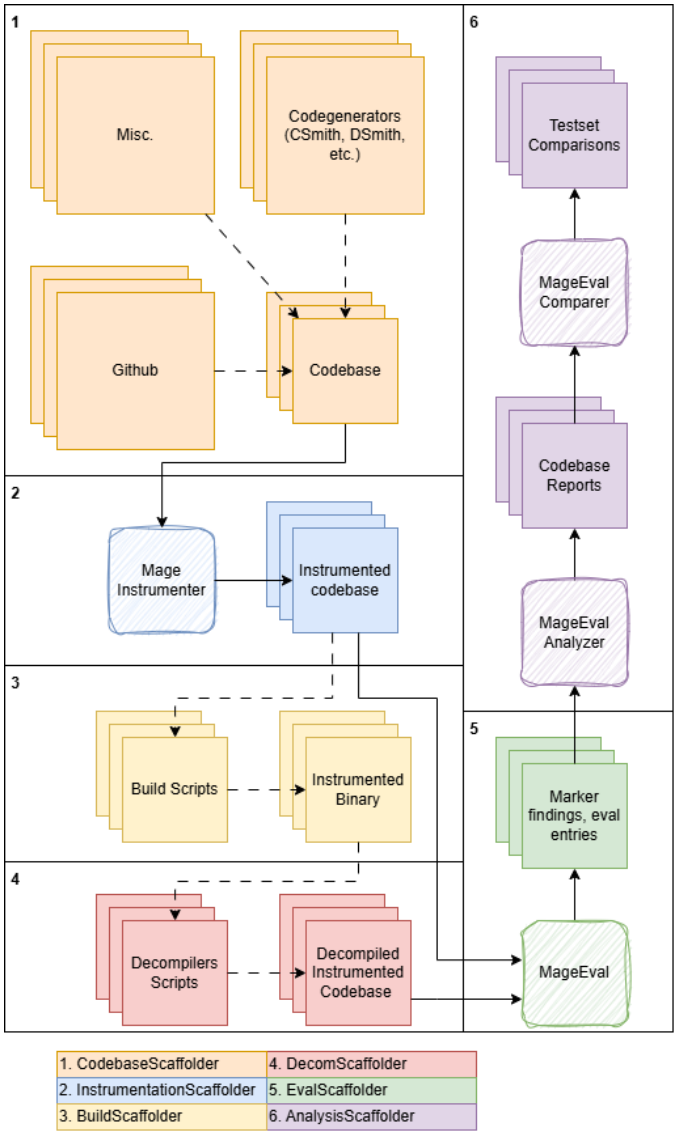


Fig. 1: Framework architecture.

InstrumentationScaffolder The task of the InstrumentationScaffolder is to facilitate the instrumentation of codebases and to store the instrumented variants within their respective separate repositories. The instrumentation markers are inserted in-place on a copy of a codebase, preserving the original while mirroring the codebase's project structure.

BuildScaffolder Each repository's build procedures are extracted to an invokable build script, where post-build instructions are added. Post-build instructions are designed to migrate the built binaries to separate related directories. Instrumented code gets built according to its repository's build script and, if necessary, with name-preserving flags so that markers

can get recognized post-decompilation. For artificial code, we invoke Clang (21.1.8) with only the seed flag and the required files it needs to link for compilation. After the compilation completes, all the produced binaries are moved to a related directory ready for decompilation.

DecomScaffolder The DecomScaffolder selects decompilation scripts for instrumented binaries and extracts the decompiled sources to a related post-decompilation directory.

EvalScaffolder The EvalScaffolder tasks MageEval with the extraction of marker findings from the oracle and decom sources, followed by the production of evaluation entries from the findings.

AnalysisScaffolder The AnalysisScaffolder first tasks MageEvalAnalyzer with producing summaries using the available findings and evaluation entries. The AnalysisScaffolder then tasks the MageEvalComparer to aggregate the summaries into collective overviews, e.g. artificial or real-world sets.

III. Benchmark Methodology

In this proof-of-concept, we measure the general recovery of language constructs. We choose concrete metrics that relate to observable differences, which is ideal for demonstrating the mapping principle. Therefore we measure the recovery in terms of statement classes. We instrument the following subset of Clang statement classes [20], that are common in source code for the demonstration:

`BinaryOperator` represents a binary operator expression.
`CaseStmt` represents cases of a switch structure.
`CallExpr` represents a function call.
`DeclStmt` represents at least one declaration.
`ForStmt` represents a for-loop structure.
`GotoStmt` represents a goto jump statement.
`IfStmt` represents a conditional if statement.
`SwitchStmt` represents a switch structure.
`WhileStmt` represents a while-loop structure.

A. Assumptions

From the specifications [15**?** , 18, 21] of the instrumentation techniques we make the following assumptions:

Markers Markers do not change program semantics and retain the order of their observable effects. Markee statements and representations of their execution statements do not change their relative order with respect to markers.

Attributes Functions that are attributed with optimization instructions, can only undergo additional transformations classified as optimization.

Preprocessor directives Translation units declared with directives (Listing 4) can only undergo additional transformations classified as optimization.

We assume all optimizations to be performed by an optimizer and that an optimizer performs no other actions. We assume compilers to adhere to the as-if rule, allowing for the possibility of modifications without changing the observable behaviour.

For compilation and decompilation we assume symbol or semantic information related to call arguments, to persist through decompilation. For decompilers we expect the decompiled code to be parsable by Clang and to be free of macros.

The asserted binary outcome is, when declaring O0 and with Mage applied, that the executable assembly represents the execution structure of the instrumented source code.

IV. Results and Discussion

For test sets, existing literature is lacking a single defined standard, as multiple data set constructions exist [3, 4, 5, 8, 9, 10, 22]. The motivations behind test set constructions vary from measuring specific effects of debug settings to evaluating decompilation within realistic representative circumstances. Data sets that strive for representative code also differ in whether they include obfuscated code. Existing test sets operate on ground truths available at various granularities, of which function-level is most prominent in recent benchmarks. There exist no data set tailored to statement-level instrumentation on code that is evaluated within the original source context of a program. Given the absence of an existing real-world test set for our heuristic, we construct a set in the proof-of-concept designed to survey for the effects of code bases on the selected baseline heuristic and the corresponding recovery metrics.

We include arbitrary "artificial" code from generators such as CSmith and "real-world" code from active GitHub repositories. We require a decompiler that produces parsable (pseudo) C output for the proof-of-concept, with no specific additional requirements. We choose a modern decompiler like Ghidra (12.1.2_PUBLIC_20260605).

We gathered code from 3 generators for artificial code; CSmith (seeds 0 - 1000) [23], DSmith (seeds 0 - 1000) [24] and Yarpgen (seeds 1 - 1000) [25], compiling each using the Clang compiler with no extra flags. For real-world code we installed the required build packages and built per repository instructions. We set the optimization level to "none" (O0) to simulate minimal conditions on decompiler performance without additional intermediate transformations. The evaluated decompiler is Ghidra [26]. Characteristics of the included codebases can be found in Appendix A. The benchmark was performed on a virtual image of `Fedora 43 Workstation` with 8 cores of `13650HX` and 8 GB RAM.

A. Statement Recovery

The following describes Ghidra's output for constructs from the targeted binaries:

- `BinaryOperator`: The number of found statements is almost 60 times the oracle amount and notably the only targeted statement that was found more often

TABLE II: Recovery statistics over the compiled test sets.

(a) Artificial

Construct	P	FP Comp.	Recovered
CallExpr	0.8419	116,530	98,107
BinaryOperator	0.0000	0	0
CaseStmt	0.0000	0	0
DeclStmt	0.0000	80,530	0
ForStmt	0.6085	44,137	26,857
GotoStmt	0.0000	0	0
IfStmt	0.8287	19,862	16,459
SwitchStmt	0.0000	0	0
WhileStmt	0.7951	3,914	3,112

(b) Real-World

Construct	P	FP Comp.	Recovered
CallExpr	0.8808	2,533,009	2,231,037
BinaryOperator	0.6638	11,079	7,354
CaseStmt	0.0058	2,240	13
DeclStmt	0.0005	2,956,450	1,440
ForStmt	0.6294	328,954	207,054
GotoStmt	0.8368	75,731	63,374
IfStmt	0.7159	3,530,785	2,527,619
SwitchStmt	0.0009	67,631	60
WhileStmt	0.4640	175,785	81,566

than its oracle counterpart in the real-world test set. None were generated in the artificial test set oracle.

- `CallExpr`: `CallExpr` recovery is well over 80% for both test sets as the top statistic.
- `CaseStmt`: Hardly any `CaseStmt` belonging to `SwitchStmt` is recovered across all sets. The artificial set did not produce any case statements. Furthermore, almost no marker finding comparisons were made—even less than `SwitchStmt`. The C grammar allows statements on the same scope as the case statements, which we use to instrument `CaseStmt`. Unreachable legal statements however, such as marker "83" and "84", are discarded by the Clang compiler even on O0 (Listing 7).

```
switch (x) {
    mage_alpha83(83);          <-- discarded on O0
    case 1:
        printf("One\n");
        break;
    mage_alpha84(84);          <-- discarded on O0
    case 2: printf("Two\n");
    mage_alpha85(85);
}
```

Listing 7: Statement removal in switch statements on O0.

- `DeclStmt`: `DeclStmt` recovery is minimal; type uncertainty is a common problem in decompilation, resulting in Ghidra outputting pseudocode without type declaration, which is interpreted as `BinaryOperator`.
- `ForStmt`: `ForStmt` recovery is moderate, with percentages above 60% for all test sets.
- `GotoStmt`: No `GotoStmt` was generated in the artificial test set oracle, thus no recovery was measured. However, in the real-world test set, recovery was over 80%.
- `IfStmt`: `IfStmt`'s recovery is reasonable, with recovery proportions over 70% across test sets.
- `SwitchStmt`: As observed with `CaseStmt`, only the real-world set yields negligible recovery. `CaseStmt` had zero occurrences in the artificial test set, corresponding to the lack of `SwitchStmt`.
- `WhileStmt`: The artificial test set exhibited a higher recovery rate compared to the real-world test set; notably, DSmith was the sole generator of `WhileStmt` in the artificial set.

If we aggregate the statement recoveries, then the recovery probabilities paint a misleading picture, suggesting that the difference might not be as substantial as it appears for the different test sets (Table III).

TABLE III: Aggregated statement recovery metrics.

Test set	P	Recovered	FP Comparisons
Artificial	0.5455	144,535	264,973
Real-World	0.5288	5,119,517	9,681,664

Results diverge considerably depending on the composition of a test set, illustrated by the performance variations across individual codebases (Table IV).

Metrics on generated code show differences even between the variants of individual codebases like CSmith. Codebases such as Radare2 stand out as outliers, indicating characteristics that evade syntactic reconstruction. The results, regardless of the chosen test set, have recovery proportions distinct to the algorithms of the whole pipeline.

B. Recovery Patterns as Indicator of Pipeline Effects and Decompiler Behavior.

Decompilers have their own production rules for translating machine code into source code, which contributes to different markee recovery compositions. We explore some of the alternate recovery instances to examine how decompiler algorithms, pipeline effects and benchmarks results intertwine through the framework.

For the aggregated markee recovery possibilities (Table V), we note `BinaryOperator` to be a frequent markee recovery alternative. Ghidra recovers expressions with preassignments to variables (Listing 8) from the test set binaries. Productions by Ghidra that lack type specifics result in constructs being recorded as a `BinaryOperator`, where Ghidra did not use a placeholder type. This result is consistently found for `ForStmt`, `WhileStmt`, and `SwitchStmt`.

For `DeclStmt` in particular, due to the loss of type information, Ghidra's productions are recovered as `BinaryOperator` expressions by Clang (Listing 9).

Notably, a `DeclStmt` containing multiple declarations can be reinterpreted as a `CallExpr` (Listing 10).

TABLE IV: Recovery proportion per construct for each codebase in percentages (%).

Codebase	CallExpr	BinaryOperator	CaseStmt	DeclStmt	ForStmt	GotoStmt	IfStmt	SwitchStmt	WhileStmt
Real-World									
AvdanOSImager	100.00	0.00	0.00	0.00	100.00	0.00	63.83	0.00	100.00
Git-2.54.0	88.15	66.35	0.00	0.10	62.94	83.97	71.59	0.00	46.39
Hashcat-7.1.2	78.42	87.50	0.00	0.00	62.43	0.00	49.32	0.00	38.60
Radare2-6.1.8	11.47	0.00	0.00	0.00	6.52	1.48	7.56	0.00	4.84
Redis-8.8.0	61.96	59.26	0.00	0.00	57.25	62.50	72.72	0.00	59.55
Sqlite-amalgamation	85.13	0.00	44.83	0.00	64.06	86.82	87.32	41.96	55.36
Tmux-3.6b	91.30	0.00	0.00	0.00	75.63	85.07	74.04	0.00	41.84
Artificial									
CSmith (Linux)	76.70	0.00	0.00	0.00	29.68	0.00	55.87	0.00	0.00
CSmith (windows)	97.39	0.00	0.00	0.00	85.63	0.00	89.24	0.00	0.00
DSmith (Linux)	98.11	0.00	0.00	0.00	0.00	0.00	83.92	0.00	74.91
DSmith (windows)	100.00	0.00	0.00	0.00	0.00	0.00	86.25	0.00	86.03
Yarpgen-codegen	54.68	0.00	0.00	0.00	41.23	0.00	72.86	0.00	0.00

```
---   oracle   ---
mage_alpha18514(18514);
if (hc_lockfile(&fp) == -1) {                <--
  hc_fclose(&fp);
  event_log_error(hashcat_ctx, "\%s: \%s",
    dictstat_ctx->filename, strerror(errno)); <--
return -1; ...
---   decom   ---
mage_alpha18514(0x4852);
iVar3 = hc_lockfile(&fp);           <-- extraction
if (iVar3 == -1) {
  hc_fclose(&fp);
  piVar4 = __errno_location();      <-- extraction
  pcVar5 = strerror(*piVar4);       <-- extraction
  event_log_error(hashcat_ctx, "\%s: \%s",
    pdVar1->filename, pcVar5);
  iVar3 = -1;}
else {...} ...
```

Listing 8: Extraction of expressions to variables.

```
---   oracle   ---
int hd = hdrsize + offset;
unsigned long llen = linelen(buffer + hd, size - hd);
---   decom   ---
hd = offset + hdrsize;
llen = linelen(buffer + hd,size - (long)hd);
```

Listing 9: `DeclStmt` typeloss leads to pseudocode `BinaryOperator` interpretation.

```
---   oracle   ---
mage_alpha5109(5109);
char c, last[] = "X;:", **argv, *value;     <--
---   decom   ---
mage_alpha5109(0x13f5);
builtin_strncpy(last,"X;:",4);              <--
```

Listing 10: Reinterpretation to function calls.

In other cases, `CaseStmt` was the most often recovered as `GotoStmt` or `LabelStmt`, alongside the regression of the encompassing `SwitchStmt`. The default and case statements could not be unified within the context of a `SwitchStmt`, possibly stemming from the lack of sufficient information for abstraction or due to a conservative algorithm.

Loss of `IfStmt` instances can occur due to compilers performing partial local optimizations even when attributes set flags to turn off local optimizations. For `IfStmt` and Clang, the LLVM-IR is accompanied by the instruction `"icmp eq ..."`, which can be removed under optimizations despite local attributes and directives declaring optimization to be disabled (Listing 11). Such effects lead to decompiled code without the `IfStmt`.

```
; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @testExpr() #0 {
  ret i32 11
}

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  %2 = call i32 @testExpr()
  %3 = icmp eq i32 %2, 11
  br i1 %3, label %4, label %6
```

Listing 11: The `icmp eq` instruction will not be present in IR with optimization enabled, despite the optnone attribute.

These productions are particular to the underlying compiler and decompiler, thus it is not unexpected for each benchmark to have the underlying algorithms reflected through their recovery distributions. The same considerations hold for inspecting attributes (Table VI), where most variance occurs for the has_callee attribute.

V. Validity and exceptions

The framework instrumentation and evaluations act solely within the source code context. There is no single compiler specification on how source code is transformed to binary, which makes the validation of the marker's core mechanic specific to the underlying compilers.

Fundamentally, the validation of this assumption and therefore the instrumentation mechanism depends on the criteria of our heuristic and compiler behavior, e.g. accounting for early diversion by compilers from source code structure, if we want to evaluate on exact construct recovery by decompilers. Our validation therefore rests on

TABLE V: Oracle markee recovered as decom markee in percentages (%). For each construct the upper row is "Artificial" and the lower row is "real-world".

Oracle	BinaryOperator	BreakStmt	CallExpr	CaseStmt	DeclStmt	DoStmt	ForStmt	GotoStmt	IfStmt	ImplicitCastExpr	LabelStmt	RecoveryExpr	ReturnStmt	SwitchStmt	WhileStmt
CallExpr	<1	0	84.2	0	0	0	0	0	0	15	0	0	0	0	0
	8.7	0	88.7	0	<1	0	0	0	<1	1.5	0	0	<1	0	0
CaseStmt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	<1	0	0	<1	0	0	0	36.1	0	0	55.9	0	7.3	0	0
DeclStmt	44.5	0	44.4	0	0	<1	7.8	0	<1	<1	<1	0	1.4	0	0
	52.3	0	44.6	0	0	0	<1	0	2.6	<1	0	0	<1	0	<1
ForStmt	13.9	0	22.7	0	0	<1	60.8	0	1.1	0	0	0	<1	0	1
	28.6	0	1.4	0	0	1.6	62.9	0	<1	0	<1	0	<1	0	4.3
GotoStmt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	2.7	2.3	<1	0	0	<1	0	83.7	<1	0	5.9	0	4.8	0	0
IfStmt	12.8	0	4.1	0	0	0	0	0	82.9	<1	0	0	0	0	0
	28.2	0	<1	0	0	0	0	0	71.6	0	0	0	<1	0	0
SwitchStmt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	48.5	0	0	0	0	0	0	0	51.1	0	0	0	<1	<1	0
WhileStmt	8.7	0	5	0	0	3.5	2.5	0	<1	0	0	0	0	0	79.5
	21.9	0	<1	0	0	11.6	17	0	<1	0	2.3	0	<1	0	46.4

the following capability as requirement, which we claim not to be the default behavior of compilers but that such behavior is possible under appropriate circumstances:

Requirement 1: There exists a circumstance under which a compiler can preserve statement semantics, execution order, and assembly characteristic of language constructs when compiling binary.

As an example, we explore to which extent Clang satisfies this requirement with assumptions based on Clang and the C language specifications. Clang's operation involves 3 phases [27]:

Frontend Involves preprocessing, parsing and conversion to intermediate representations (LLVM-IR).

Middle-end Phase that handles optimizations.

Backend Phase that generates the final code.

Practically this results in 3 accessible representations without optimization: AST, LLVM-IR and assembly. Clang's AST strives to be "faithful" to the original source code [28]. Each subsequent phase after provides no guarantees and acts according to the as-if rule, i.e. any changes are allowed as long as the resulting behavior is unchanged [29]. Clang performs minor "optimizations" under O0 for unreachable code, e.g. markers 83 and 84 are present in the AST but absent from the LLVM-IR representation (Listing 7). For code that compiles with optimizations, attributed functions and the header allow some minor local optimizations to occur, despite the specifications in the Clang documentation [21].

Declarations without assignments do not translate directly to assembly in Clang [29], and Clang reinterprets such declarations to the start of functions in the LLVM-IR. Clang's LLVM-IR approach to single static assignment (SSA) also extracts expressions to separate assignments.

Clang preserves the order of observable effects [29], which include Mage's markers (Section II-B). Therefore markers keep their relative order.

Mage adds a constraint through barriers to prevent the reordering of assembly generated for statements across markers [**?**].

Finally, Mage adds a second constraint through directives and attributes, which suppress the removal and transformation of code beyond basic translation (Listing 3 and 4).

Under these conditions, Clang satisfies the requirement with at least three exceptions from its IR implementation details. First, declarations without assignments are relocated to the start of a function in the IR. Second, unreachable statements are removed before binary emission. Third, expressions get preassigned to variables in IR. Such exceptions define the valid domain of the benchmark: constructs affected by exceptions are interpreted as pipeline observations, and all others as decompiler observations.

TABLE VI: Recovery proportions for attributes (in %) for codebases. "-" = construct not present.

Codebase	BinaryOperator	CallExpr		DeclStmt	ForStmt			IfStmt		SwitchStmt		WhileStmt
	operator	has_callee	num_args	single_decl	has_init	has_cond	has_incr	has_cond	has_else	has_cond	has_cases	has_cond
Artificial												
Aggregated	-	89.0	51.9	-	99.6	100.0	99.3	100.0	96.6	-	-	100.0
CSmith (Linux)	-	100.0	99.8	-	100.0	100.0	100.0	100.0	89.1	-	-	-
CSmith (Windows)	-	100.0	4.3	-	100.0	100.0	100.0	100.0	98.6	-	-	-
DSmith (Linux)	-	100.0	100.0	-	-	-	-	100.0	98.7	-	-	100.0
DSmith (Windows)	-	100.0	0.0	-	-	-	-	100.0	99.0	-	-	100.0
Yarpgen-codegen	-	20.0	93.5	-	93.4	100.0	86.8	100.0	79.7	-	-	-
Real-world												
Aggregated	100.0	75.0	99.9	100.0	97.5	100.0	99.6	100.0	83.4	100.0	100.0	100.0
AvdanOSImager	-	100.0	100.0	-	100.0	100.0	100.0	100.0	36.7	-	-	100.0
Git-2.54.0	100.0	75.1	99.9	100.0	97.5	100.0	99.6	100.0	83.4	-	-	100.0
Hashcat-7.1.2	100.0	56.1	98.6	-	99.1	100.0	99.1	100.0	76.7	-	-	100.0
Radare2-6.1.8	-	90.9	98.2	-	100.0	100.0	100.0	100.0	78.4	-	-	100.0
Redis-8.8.0	100.0	34.3	99.6	-	85.3	100.0	100.0	100.0	86.7	-	-	100.0
Sqlite-amalgamation	-	49.1	98.2	-	98.8	100.0	100.0	100.0	86.8	100.0	100.0	100.0
Tmux-3.6b	-	75.3	100.0	-	98.5	100.0	100.0	100.0	82.1	-	-	100.0

VI. Limitations

For source code context and macros, a mechanism for arbitrary text replacement, markers can lead to invalid code when combined with macros. The current implementation uses a conservative approach to source code instrumentation related to macros; as a result, potentially interesting statements can remain unmarked. Compilers can rearrange or remove code even without optimization; thus, domain knowledge of specific compilers is necessary when interpreting results. In addition compiler implementations can still perform minor local optimizations despite attributes and headers declarations turning optimizations off locally. Within optimizations, intermediate representations and their representative source code correspond to distinct ground truths, e.g. inlined versus non-inlined function. In addition, the presence of markers disrupts optimization productions by artificially introducing optimization barriers required for ground truth instrumentation. Therefore, separate benchmarks are required for this category that account for such differences and effects.

Repositories with custom build scripts can be unpredictable, where modifying flags is nontrivial. As a consequence, manual intervention is potentially required, which can obstruct use of the framework by independent parties. For decompiler outputs, the Clang AST parser does not require the syntax to be completely correct as it can buffer these to "recovery expressions". It therefore assumes decompilers to output sufficiently parsable (pseudo) C by Clang's standards.

In instrumentation, the presence of markers in decompiler output can have unwanted effects on existing metrics used within literature, such as the edit similarity.

Test sets in benchmarks can include repositories that have influence on benchmark outcomes. Current literature lacks research on what a test set should exactly constitute, consequently sets are chosen at random from sources. However we have demonstrated that the composition of a set can influence results, therefore the lack of such a standard adds fundamental uncertainty to the representativeness of benchmarks. The proof-of-concept is limited to O0 and its baseline heuristic that provides basic information. In addition, this specific information is sensitive to compiler algorithms regarding the instrumented binary, e.g. exceptions (Section V).

Each process involved in the build of the final binary can disrupt or contribute to exceptions within the satisfaction of Mage's requirement (Section V), this can be mitigated by having a compiler that complies in full with the requirements. Despite instrumentations taking place on source code level, the compilation algorithms have to be taken into consideration for compliance to Mage's requirements.

VII. Related Work

Transparent benchmark metrics and methodologies are some of the core components of benchmarking. We started our work with the intent of making benchmarks more intuitive and to have the option to drill down into details to enable more actionable feedback for developers. We review related work on benchmarks with an emphasis on such feedback.

Aggregated metrics, such as R2I [6], condense their subfeatures into a single metric. R2I's subfeatures themselves are weighted subscores from surveys. It is unclear how such scores can be utilized beyond generic indicators even when using its subfeatures as guidance towards areas of attention.

Specific metrics like edit similarity (Levenshtein distance) are sometimes applied to arbitrary binary conditions [3]. However, the Levenshtein distance can produce misleading penalizing scores for readability when applied across equivalent decompiler outputs for the same binary with and without debug symbol information, hence requiring further distinction in readability for structure and human naming context.

By specifying and applying distinct concrete metrics more selectively, developers can choose aspects of performance to group and inspect. Such selective grouping is currently preemptively done by researchers under other collective metrics with possible overlaps in features, such as "URAF readability" [5], "L1-5" readability [5], "memory layout abuse" [22] and "memory layout". For broad categories, existing benchmark aggregations give impressions of overlaps and differences in metrics that detract from clarity regarding the completeness of benchmarks and their interpretations in relation to each other.

Test sets of existing benchmarks [3, 4, 8, 9, 22, 24] are currently chosen arbitrarily, where random selection is assumed to be representative. However, there is preliminary evidence (Table IV) that sets should avoid repetition of constructions that can skew scores.

The current state of decompiler benchmarking lacks standard specifications for benchmarks and their metrics [2, 30]. By realizing criteria directly related to source code, we contribute to transparent benchmarks for decompilers.

VIII. Conclusion

Existing benchmarks produce reports that can benefit from more diagnostically oriented outputs. To supplement holistic reports, we started our contribution by targeting concrete aspects of decompiler performance at the source level, which is the restoration of language constructs with no intermediate optimization passes. We demonstrated benchmarks on exact and alternative recoveries that reflect how algorithms handle distinct constructs. We highlighted the downside of holistic statistics, e.g. aggregated statement recovery reporting minimal differences between test sets, in contrast to more nuanced reporting. Within the results, patterns hinted at how compiler and decompiler algorithms corresponded with benchmark results and reflect their characteristics, encouraging specific directions for investigation. In addition, we were able to detect how distinct tests within a set can influence outcomes and the need for a representative standard test set composition. Throughout the inspection we demonstrated the potential of our artifact for instrumenting arbitrary code and its use for benchmark investigations. Furthermore, we showcased how this form of concrete benchmarking can further expose transparent metrics.

Experts and developers rely on benchmarks to accurately evaluate decompiler performance and to convey a clear story. Metrics signal and guide developers towards aspects for future improvement. A move towards more diagnostic forms of benchmarking assists experts and developers in using such results productively.

A. Future Work

From our findings we identify two prospective areas for research. Granular benchmarking (with Mage) and benchmarking standards. The proof-of-concept demonstrated how benchmarks are influenced by the underlying components in the pipeline. Earlier preliminary results served as a demonstration of the framework's capabilities. For comprehensive results, targeted efforts are required to benchmark decompilers under controlled conditions. Mage's mechanisms enable control over the optimization level through the source code itself. In addition, metrics on constructs can be expanded to include semantics, where type details and properties such as side-effects can be measured. This requires the inclusion of libraries during analysis in order to resolve all semantics. Furthermore, the heuristics can be expanded to include existing or new metrics, e.g. criteria from Dramko et al. [22].

Existing metrics can be further refined by investigating the robustness and effects of markers in relation to its pipeline components. For source code, macros can contain interesting code to benchmark. On the other hand, the absence of markers can have undiscovered potential uses beyond detecting dead code elimination [11] and pipeline characteristics (Section V).

Standards and definitions for benchmarking are areas that are underdeveloped. Granular benchmarks such as Mage's can detect the influence of test sets on metrics. By investigating test sets in more detail, progress can be made towards sets to systematically construct more suitable code for evaluation.

References

[1] C. Cifuentes, *Reverse compilation techniques*. Queensland University of Technology, Brisbane, 1994.

[2] L. M. Weber, W. Saelens, R. Cannoodt, C. Soneson, A. Hapfelmeier, P. P. Gardner, A.-L. Boulesteix, Y. Saeys, and M. D. Robinson, "Essential guidelines for computational method benchmarking," *Genome biology*, vol. 20, no. 1, p. 125, 2019.

[3] H. Tan, X. Tian, H. Qi, J. Liu, Z. Gao, S. Wang, Q. Luo, J. Li, and Y. Zhang, "Decompilebench: Million-scale binary-source function pairs for real-world binary decompilation," *arXiv preprint arXiv:2505.12668*, 2025.

[4] Z. Gao, Y. Cui, H. Wang, S. Qin, Y. Wang, Z. Bolun, and C. Zhang, "Decompilebench: A comprehensive benchmark for evaluating decompilers in real-world scenarios," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 23 250–23 267.

[5] P. Liu, Y. Huang, J. Chi, P. Di, and Y. Jiang, "Codefuse-debench: An empirical study on readability, recompilability, and functionality," 2026. [Online]. Available: https://arxiv.org/abs/2605.29490

[6] H. Eom, D. Kim, S. Lim, H. Koo, and S. Hwang, "R2i: A relative readability metric for decompiled code,"

Proceedings of the ACM on Software Engineering, vol. 1, no. FSE, pp. 383–405, 2024.

[7] V. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," in *Soviet physics-doklady*, vol. 10, no. 8, 1966, pp. 707–710.

[8] J. Kline, "A framework for assessing decompiler inference accuracy of source-level program constructs," Master's thesis, University of Kansas, 2022.

[9] K. v. G. Jaap van den Bos and K. Reijer, "Deb'm: A benchmarking platform for decompilers," 2024.

[10] M. Zou, A. Khan, R. Wu, H. Gao, A. Bianchi, and D. J. Tian, "D-helix: A generic decompiler testing framework using symbolic differentiation," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 397–414.

[11] T. Theodoridis, M. Rigger, and Z. Su, "Finding missed optimizations through the lens of dead code elimination," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 697–709.

[12] DWARF Debugging Information Format Committee, *DWARF Debugging Information Format, Version 5*, Feb. 2017, available at https://dwarfstd.org/doc/DWARF5.pdf. [Online]. Available: https://dwarfstd.org/

[13] G. A. Di Luna, D. Italiano, L. Massarelli, S. Österlund, C. Giuffrida, and L. Querzoni, "Who's debugging the debuggers? exposing debug information bugs in optimized binaries," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1034–1045. [Online]. Available: https://doi.org/10.1145/3445814.3446695

[14] cppreference.net contributors, "Attribute specifier sequence (C23): Attribute testing," cppreference.net, July 2026, accessed: 2026-07-22. [Online]. Available: https://cppreference.net/c/language/attributes.html#Attribute_testing

[15] *Attributes in Clang*, The Clang Team, 2026, accessed: July 30, 2026. [Online]. Available: https://clang.llvm.org/docs/AttributeReference.html

[16] *Using the GNU Compiler Collection (GCC): Common Attributes*, Free Software Foundation, Inc., 2026, accessed: July 30, 2026. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Common-Attributes.html

[17] Free Software Foundation, *The C Preprocessor*, Free Software Foundation, Inc., 2026. [Online]. Available: https://gcc.gnu.org/onlinedocs/cpp/

[18] *Using the GNU Compiler Collection (GCC): Extended Asm - Assembler Instructions with C Expression Operands*, Free Software Foundation, Inc., accessed: 2026-09-30. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Extended-Asm.html#Clobbers-and-Scratch-Registers

[19] Clang Team, "Clang language compatibility: Inline assembly," https://clang.llvm.org/compatibility.html#inline-asm, 2026, accessed: 2026-10-15.

[20] LLVM Project. (2026) clang: Class index. Generated via Doxygen. Accessed: 2026-07-20. [Online]. Available: https://clang.llvm.org/doxygen/Stmt_8h_source.html

[21] *Attributes in Clang: optnone*, The Clang Team, 2026, online documentation; accessed 2026-09-30. [Online]. Available: https://clang.llvm.org/docs/AttributeReference.html#optnone

[22] L. Dramko, J. Lacomis, E. J. Schwartz, B. Vasilescu, and C. Le Goues, "A taxonomy of c decompiler fidelity issues," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 379–396.

[23] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2011, pp. 283–294.

[24] Y. Cao, R. Zhang, R. Liang, and K. Chen, "Evaluating the effectiveness of decompilers," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 491–502.

[25] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for c and c++ compilers with yarpgen," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020. [Online]. Available: https://doi.org/10.1145/3428264

[26] National Security Agency, "Ghidra: Software Reverse Engineering Framework," https://ghidra-sre.org/, 2026, accessed: 2026-01-30.

[27] *Clang Compiler User's Manual*, Clang, 2026. [Online]. Available: https://clang.llvm.org/docs/UsersManual.html

[28] *"Clang" CFE Internals Manual*, Clang, 2026, version 24.0.0git. [Online]. Available: https://clang.llvm.org/docs/InternalsManual.html

[29] aengelke, "Adherence to source code representation within the difference phases of the compilation process," LLVM Discussion Forums, 2026. [Online]. Available: https://discourse.llvm.org/t/adherence-to-source-code-representation-within-the-difference-phases-of-the-compilation-process/91766/4

[30] J. Zhan, "Call for establishing benchmark science and engineering," *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, vol. 1, no. 1, p. 100012, 2021.

Appendix

A. Codebase Characteristics

TABLE VII: Characteristics of selected "real-world" and "artificial" codebases.

Codebase	Arch	Bits	Binary	Class	Compiler	Endian	Machine	OS
AvdanOSImager	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	AMD x86-64	linux
git-2.54.0	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	AMD x86-64	linux
hashcat-7.1.2	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	AMD x86-64	linux
radare2-6.1.8	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	AMD x86-64	linux
redis-8.8.0	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	AMD x86-64	linux
sqlite-amalgamation-3530300	x86	64	elf	ELF64	GCC: (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7) clang version 21.1.8 (Fedora 21.1.8-4.fc43)	little	AMD x86-64	linux
tmux-3.6b	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	AMD x86-64	linux
csmith	x86	64	elf	ELF64	GCC: (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7) clang version 21.1.8 (Fedora 21.1.8-4.fc43)	little	AMD x86-64	linux
csmith	x86	32	pe	PE32	LLVM MinGW[1]	little	i386	windows
dsmith[2]	x86	64	elf	ELF64	GCC: (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7) clang version 21.1.8 (Fedora 21.1.8-4.fc43)	little	AMD x86-64	linux
dsmith[2]	x86	32	pe	PE32	LLVM MinGW[1]	little	i386	windows
yarpgen-codegen	x86	64	elf	ELF64	GCC: (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7) clang version 21.1.8 (Fedora 21.1.8-4.fc43)	little	AMD x86-64	linux

[1]Compiler: GCC (15.1.1 20250521 (Fedora MinGW 15.1.1-1.fc43)).
[2]Modification made to DSmith's source.py to allow seeds.